

线性滤波实验报告

高斯金字塔 · 噪声生成 · 平滑滤波 · 去噪处理

章屹倬 学号：231880366

2026 年 6 月

计算机视觉课程作业（1）

提交日期：2026 年 6 月 2 日

摘 要

本实验报告围绕图像线性滤波展开，使用 Python 3 与 OpenCV 4.8 实现了五大核心模块：（1）高斯金字塔的逐层构建与多尺度分析；（2）高斯噪声与椒盐噪声的数学模型与生成；（3）均匀平滑（均值滤波）与高斯平滑的空间域滤波对比；（4）基于高斯平滑与中值滤波的去噪对比实验；（5）综合可视化分析。

实验以 2268×4032 分辨率的真实照片为测试图像，系统考察了线性滤波器（Box Filter、Gaussian Filter）与非线性滤波器（Median Filter）在不同噪声模型下的表现差异。结果表明：线性滤波器对高斯噪声具有良好抑制效果，但对椒盐噪声无效甚至产生伪影；中值滤波作为非线性方法，对脉冲噪声（椒盐噪声）具有卓越的剔除能力，同时能较好地保持边缘信息。实验揭示了空间域滤波中“噪声抑制”与“边缘保留”之间的根本性权衡，为后续学习更高级的图像处理技术奠定了基础。

关键词：线性滤波；高斯金字塔；高斯噪声；椒盐噪声；均值滤波；高斯平滑；中值滤波；去噪

目录

1	引言	5
1.1	实验背景	5
1.2	实验目的	5
1.3	实验环境	5
2	算法原理	6
2.1	高斯金字塔 (Gaussian Pyramid)	6
2.1.1	数学定义	6
2.1.2	参数设置	6
2.2	噪声模型	6
2.2.1	高斯噪声 (Gaussian Noise)	6
2.2.2	椒盐噪声 (Salt-and-Pepper Noise)	7
2.3	空间域平滑滤波	7
2.3.1	均匀平滑 (Box Filter / 均值滤波)	7
2.3.2	高斯平滑 (Gaussian Filter)	7
2.3.3	中值滤波 (Median Filter)	8
2.4	去噪评价方法	8
3	代码实现	9
3.1	整体架构	9
3.2	详细代码 (含注释)	9
4	实验结果与分析	15
4.1	高斯金字塔	15
4.2	噪声生成	15
4.3	空间域平滑滤波	16
4.4	去噪处理	17
4.5	综合分析	18
5	讨论	19
5.1	线性 vs 非线性滤波	19
5.2	噪声抑制与边缘保留的权衡	19
5.3	参数敏感性分析	19
5.4	实验局限与改进方向	20
6	总结	21
A	完整源代码 (main.py)	22

B 输出文件清单	28
C 运行说明	29

1 引言

1.1 实验背景

图像滤波（Image Filtering）是数字图像处理中最基础也是最重要的操作之一，广泛应用于图像去噪、特征提取、图像增强、边缘检测等任务。其核心思想是利用像素邻域信息对每个像素值进行重新计算，从而改变图像的频率特性。

在频率域视角下，图像可以被分解为低频分量（平滑区域、缓慢变化的亮度）和高频分量（边缘、纹理、噪声）。**线性滤波**通过卷积核对图像进行加权求和，其数学本质是线性时不变（LTI）系统的二维推广。

1.2 实验目的

本实验旨在通过动手实现，深入理解以下核心概念与技术：

- 1. 理解高斯金字塔的多尺度表示原理，掌握 `cv2.pyrDown()` 的使用
- 2. 掌握高斯噪声和椒盐噪声的数学模型与生成方法
- 3. 对比均匀平滑（Box Filter）与高斯平滑（Gaussian Filter）的滤波特性
- 4. 分析线性滤波（高斯平滑）与非线性滤波（中值滤波）对不同噪声的去噪效果
- 5. 培养图像处理实验的对比分析能力与科学报告撰写能力

1.3 实验环境

表 1: 实验环境配置

组件	版本/说明
操作系统	Windows 11 Home China
编程语言	Python 3.8+
OpenCV	opencv-python $\geq$ 4.8.0
NumPy	$\geq$ 1.24.0
Matplotlib	$\geq$ 3.7.0
IDE	Visual Studio Code
测试图像	2268×4032 真实照片（灰度图）

2 算法原理

2.1 高斯金字塔 (Gaussian Pyramid)

高斯金字塔是图像多尺度表示的基础数据结构，通过反复进行高斯平滑与下采样构建。

2.1.1 数学定义

设原始图像为 G_0 （金字塔第 0 层），则第 $k+1$ 层由第 k 层通过以下操作得到：

$$G_{k+1}(i, j) = \sum_{m=-2}^2 \sum_{n=-2}^2 w(m, n) \cdot G_k(2i + m, 2j + n) \quad (1)$$

其中 $w(m, n)$ 为 5×5 高斯核，其权重由二维高斯分布确定：

$$w(m, n) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{m^2 + n^2}{2\sigma^2}\right) \quad (2)$$

`cv2.pyrDown()` 的内部实现即为：先使用默认 5×5 高斯核（ $\sigma \approx 1.0$ ）进行平滑以消除高频混叠，再隔行隔列进行下采样，使输出图像分辨率在宽和高上各缩减为原来的 $1/2$ 。

2.1.2 参数设置

- 金字塔层数： $L = 4$
- 各层分辨率： $L0 = 2268 \times 4032$, $L1 = 1134 \times 2016$, $L2 = 567 \times 1008$, $L3 = 284 \times 504$

2.2 噪声模型

2.2.1 高斯噪声 (Gaussian Noise)

高斯噪声又称正态噪声，是电子设备热噪声、传感器暗电流噪声等物理过程的自然模型。每个像素的噪声值独立同分布于正态分布：

$$n(x, y) \sim \mathcal{N}(\mu, \sigma^2) \quad (3)$$

加噪后的像素值为：

$$I_{\text{noisy}}(x, y) = \text{clip}_{[0, 255]} \left(I(x, y) + n(x, y) \right) \quad (4)$$

参数设置：均值 $\mu = 0$ ，标准差 $\sigma = 25$ 。由于图像像素值范围为 $[0, 255]$ ， $\sigma = 25$ 约对应 10% 的噪声水平，能产生明显的颗粒感但不至于完全淹没图像内容。

2.2.2 椒盐噪声 (Salt-and-Pepper Noise)

椒盐噪声又称脉冲噪声 (Impulse Noise)，模拟信号传输过程中的数据丢失或传感器坏点。其数学模型为：

$$I_{\text{noisy}}(x, y) = \begin{cases} 255 & \text{概率 } p/2 \quad (\text{盐噪声/白点}) \\ 0 & \text{概率 } p/2 \quad (\text{椒噪声/黑点}) \\ I(x, y) & \text{概率 } 1 - p \quad (\text{保持不变}) \end{cases} \quad (5)$$

参数设置：噪声密度 $p = 0.05$ ，即 5% 的像素被随机置为纯白或纯黑。

2.3 空间域平滑滤波

2.3.1 均匀平滑 (Box Filter / 均值滤波)

均匀平滑是最简单的线性滤波器，将卷积核内所有像素赋予相同权重：

$$g(x, y) = \frac{1}{K^2} \sum_{i=-k}^k \sum_{j=-k}^k f(x+i, y+j) \quad (6)$$

其中 $K = 2k + 1 = 5$ 为核大小。该滤波器等价于一个 5×5 的矩形窗与图像做二维卷积，每个邻域像素对结果的贡献相等。虽然实现简单，但对边缘和噪声“一视同仁”，导致边缘区域的模糊较为严重。

2.3.2 高斯平滑 (Gaussian Filter)

高斯平滑使用二维高斯函数作为卷积核的权重分布：

$$g(x, y) = \sum_{i=-k}^k \sum_{j=-k}^k G(i, j, \sigma) \cdot f(x+i, y+j) \quad (7)$$

$$G(i, j, \sigma) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{i^2 + j^2}{2\sigma^2}\right) \quad (8)$$

高斯核具有以下优良性质：

1. 旋转对称性：各向同性，不偏好特定方向的边缘
2. 可分离性：二维高斯可分解为两个一维高斯的乘积，计算效率更高
3. 中心权重最大：越靠近中心像素的邻域贡献越大，更符合人眼视觉特性

参数设置：核大小 5×5 ， $\sigma = 1.0$ 。

2.3.3 中值滤波 (Median Filter)

中值滤波是一种非线性滤波方法，它取邻域像素值排序后的中位数替代中心像素：

$$g(x, y) = \text{median} \left\{ f(x + i, y + j) \mid i, j \in [-k, k] \right\} \quad (9)$$

中值滤波的核心优势在于对脉冲噪声（椒盐噪声）的处理：噪声像素值（0 或 255）作为极端值在排序后总是位于两端，中位数必然取自正常的邻域像素，因此噪声可以被精准剔除而不影响周围像素。

参数设置：核大小 5×5 （`cv2.medianBlur` 要求核为奇数且大于 1）。

2.4 去噪评价方法

本实验采用**主观视觉评价**（定性分析），主要考察以下维度：

1. **噪声残留程度：**去噪后平坦区域的噪声是否被有效抑制
2. **边缘保持能力：**物体轮廓、纹理细节是否清晰保留
3. **伪影引入：**滤波过程是否引入了新的视觉缺陷（如块状伪影、涂抹痕迹）
4. **方法适配性：**同一滤波器对不同类型噪声的处理效果差异

3 代码实现

3.1 整体架构

程序共 277 行 Python 代码，按照功能划分为 7 个模块，主流程依次调用各模块完成实验。

表 2: 代码模块架构

模块	函数	对应 OpenCV 接口
图像加载	load_image()	cv2.imread, cv2.cvtColor
高斯金字塔	gaussian_pyramid()	cv2.pyrDown
噪声生成	add_gaussian_noise()	np.random.normal
	add_salt_pepper_noise()	np.random.randint
平滑滤波	uniform_smoothing()	cv2.blur
	gaussian_smoothing()	cv2.GaussianBlur
	median_filtering()	cv2.medianBlur
可视化	show_pyramid()	—
	show_noise_results()	—
	show_smoothing_results()	—
	show_denoising_results()	—
综合分析	comprehensive_analysis()	—
主流程	main()	—

3.2 详细代码（含注释）

图像加载模块

Listing 1: 图像加载函数

```
1 def load_image(path, gray=True):
2     """加载图像，可选转为灰度图"""
3     img = cv2.imread(path)
4     if img is None:
5         raise FileNotFoundError(f"无法加载图像: {path}")
6     img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
7     if gray:
8         img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
9     return img
```

说明：OpenCV 默认以 BGR 通道顺序读取图像。为与 Matplotlib 兼容，先将 BGR 转换为 RGB，若参数 `gray=True` 则进一步转换为单通道灰度图——后续所有滤波操作均在灰度域进行，避免通道分离与合并的额外开销。

高斯金字塔

Listing 2: 高斯金字塔构建

```
1 def gaussian_pyramid(img, levels=4):
2     """
3     构建高斯金字塔
4     每一层通过高斯模糊 + 下采样得到
5     """
6     pyramid = [img]
7     current = img.copy()
8     for i in range(levels - 1):
9         current = cv2.pyrDown(current) # 高斯平滑 + 隔行隔列下采样
10        pyramid.append(current)
11    return pyramid
```

原理：`cv2.pyrDown()` 内部执行两步操作——先用 5×5 高斯核对当前层进行卷积平滑以消除高频混叠伪影，然后丢弃所有偶数行和偶数列（即 $2 \times$ 下采样）。循环 3 次后得到 4 层金字塔，每层分辨率精确减半。

噪声生成

Listing 3: 高斯噪声生成

```
1 def add_gaussian_noise(img, mean=0, sigma=25):
2     """添加高斯噪声"""
3     noise = np.random.normal(mean, sigma, img.shape).astype(np.float32)
4     )
5     noisy = img.astype(np.float32) + noise
6     noisy = np.clip(noisy, 0, 255).astype(np.uint8)
7     return noisy
```

实现细节：先生成与图像同尺寸的独立正态分布随机数矩阵，以 `float32` 精度与原始图像相加后裁剪至 `[0, 255]` 再转回 `uint8`。使用 `float32` 中间表示是为了避免加法溢出和精度损失。

Listing 4: 椒盐噪声生成

```
1 def add_salt_pepper_noise(img, prob=0.05):
2     """添加椒盐噪声"""
```

```

3     noisy = img.copy()
4     h, w = img.shape
5     n_pixels = h * w
6
7     # 盐噪声（白色）— prob/2 的像素置为 255
8     n_salt = int(n_pixels * prob / 2)
9     salt_coords = (np.random.randint(0, h, n_salt),
10                   np.random.randint(0, w, n_salt))
11     noisy[salt_coords] = 255
12
13     # 椒噪声（黑色）— prob/2 的像素置为 0
14     n_pepper = int(n_pixels * prob / 2)
15     pepper_coords = (np.random.randint(0, h, n_pepper),
16                     np.random.randint(0, w, n_pepper))
17     noisy[pepper_coords] = 0
18
19     return noisy

```

实现细节：椒盐噪声生成使用随机坐标索引而非逐像素遍历，时间复杂度为 $O(pN)$ 而非 $O(N)$ （ N 为总像素数），在 $p = 0.05$ 时效率提升约 20 倍。盐和椒各占 $p/2$ ，确保黑白噪点数量均衡。

平滑滤波

Listing 5: 三种滤波函数

```

1 def uniform_smoothing(img, kernel_size=5):
2     """均匀平滑（均值滤波）"""
3     return cv2.blur(img, (kernel_size, kernel_size))
4
5 def gaussian_smoothing(img, kernel_size=5, sigma=1.0):
6     """高斯平滑"""
7     return cv2.GaussianBlur(img, (kernel_size, kernel_size), sigma)
8
9 def median_filtering(img, kernel_size=5):
10    """中值滤波"""
11    return cv2.medianBlur(img, kernel_size)

```

说明：三种滤波均封装为统一接口，便于后续在去噪流程中按需调用。参数 `kernel_size=5` 为实验统一设定。

可视化函数

可视化函数族(`show_pyramid`, `show_noise_results`, `show_smoothing_results`, `show_denoising_results`) 使用 Matplotlib 的 `subplots` 布局, 将多幅图像整合到同一画布中, 方便直观对比。所有输出图像以 150 DPI 保存为 PNG 格式。中文字体通过以下配置保证正常渲染:

Listing 6: 中文字体配置

```
1 from matplotlib import rcParams
2 rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu
   Sans']
3 rcParams['axes.unicode_minus'] = False
```

综合分析

Listing 7: 综合分析 3×4 子图布局

```
1 def comprehensive_analysis(original, gauss_noisy, sp_noisy,
2                             uniform, gauss_smooth,
3                             gs_on_gauss, median_on_gauss,
4                             gs_on_sp, median_on_sp):
5     """综合分析大图 — 3行4列布局"""
6     fig = plt.figure(figsize=(18, 12))
7
8     positions = [
9         (original, '原始图像', 0),
10        (uniform, '均匀平滑 (5x5)', 1),
11        (gauss_smooth, '高斯平滑 (5x5, sigma=1)', 2),
12        # 第 3 列留空, 用于视觉分组
13        (gauss_noisy, '高斯噪声 (sigma=25)', 4),
14        (gs_on_gauss, '高斯平滑去噪', 5),
15        (median_on_gauss, '中值滤波去噪', 6),
16        (sp_noisy, '椒盐噪声 (p=0.05)', 8),
17        (gs_on_sp, '高斯平滑去噪', 9),
18        (median_on_sp, '中值滤波去噪', 10),
19    ]
20
21    for img, title, idx in positions:
22        ax = fig.add_subplot(3, 4, idx + 1)
23        ax.imshow(img, cmap='gray')
24        ax.set_title(title, fontsize=10)
25        ax.axis('off')
```

```
27     # 左侧行标签
28     fig.text(0.02, 0.78, '平滑滤波', fontsize=12,
29             fontweight='bold', va='center', rotation=90)
30     fig.text(0.02, 0.50, '高斯噪声\n去噪对比', fontsize=12,
31             fontweight='bold', va='center', rotation=90)
32     fig.text(0.02, 0.22, '椒盐噪声\n去噪对比', fontsize=12,
33             fontweight='bold', va='center', rotation=90)
34
35     fig.suptitle('线性滤波综合分析', fontsize=16, fontweight='bold')
36     plt.tight_layout(rect=[0.05, 0, 1, 0.96])
37     return fig
```

布局设计：3 行 × 4 列布局，第 3 列故意留空以形成视觉分组。第一行展示平滑滤波对比（原始 / 均匀 / 高斯），第二行展示高斯噪声去噪对比，第三行展示椒盐噪声去噪对比。左侧添加旋转行标签，增强可读性。

主流程

Listing 8: 主流程

```
1 def main(image_path):
2     img = load_image(image_path, gray=True)
3
4     # 1. 高斯金字塔
5     pyramid = gaussian_pyramid(img, levels=4)
6     fig1 = show_pyramid(pyramid)
7     fig1.savefig('output_1_pyramid.png', dpi=150, bbox_inches='tight')
8
9     # 2. 噪声生成
10    img_gaussian_noise = add_gaussian_noise(img, sigma=25)
11    img_sp_noise = add_salt_pepper_noise(img, prob=0.05)
12    fig2 = show_noise_results(img, img_gaussian_noise, img_sp_noise)
13    fig2.savefig('output_2_noise.png', dpi=150, bbox_inches='tight')
14
15    # 3. 平滑滤波
16    img_uniform = uniform_smoothing(img, kernel_size=5)
17    img_gaussian_smooth = gaussian_smoothing(img, kernel_size=5, sigma
18                                           =1.0)
19    fig3 = show_smoothing_results(img, img_uniform,
20                                img_gaussian_smooth)
21    fig3.savefig('output_3_smoothing.png', dpi=150, bbox_inches='tight')
22
23    # 4. 去噪处理
```

```
22     gs_on_gaussian = gaussian_smoothing(img_gaussian_noise, 5, 1.0)
23     median_on_gaussian = median_filtering(img_gaussian_noise, 5)
24     gs_on_sp = gaussian_smoothing(img_sp_noise, 5, 1.0)
25     median_on_sp = median_filtering(img_sp_noise, 5)
26     fig4 = show_denoising_results(
27         img_gaussian_noise, gs_on_gaussian, median_on_gaussian,
28         img_sp_noise, gs_on_sp, median_on_sp)
29     fig4.savefig('output_4_denoising.png', dpi=150, bbox_inches='tight')
30
31     # 5. 综合分析
32     fig5 = comprehensive_analysis(
33         img, img_gaussian_noise, img_sp_noise,
34         img_uniform, img_gaussian_smooth,
35         gs_on_gaussian, median_on_gaussian,
36         gs_on_sp, median_on_sp)
37     fig5.savefig('output_5_analysis.png', dpi=150, bbox_inches='tight')
    )
```

4 实验结果与分析

4.1 高斯金字塔



图 1: 高斯金字塔（4 层，逐层下采样）

结果分析：从图 1 可以清楚地观察到多尺度表示的特性：

1. **分辨率递减**：L0 为原始 2268×4032 分辨率，L1 降至 1134×2016 ，L2 为 567×1008 ，L3 仅 284×504 ——每层的像素总数约为上一层的 $1/4$ 。
2. **信息量递减**：随着层数增加，图像细节逐渐消失。发丝、衣物褶皱等高频纹理在 L2 已基本不可分辨，L3 仅保留人脸的整体轮廓和明暗分布。这体现了低通滤波的能量集中效应——低频分量包含了图像的绝大部分视觉信息。
3. **无混叠伪影**：由于 `cv2.pyrDown()` 在下采样前先执行高斯平滑，有效抑制了奈奎斯特频率以上的分量，各层均未出现摩尔纹或锯齿。

4.2 噪声生成

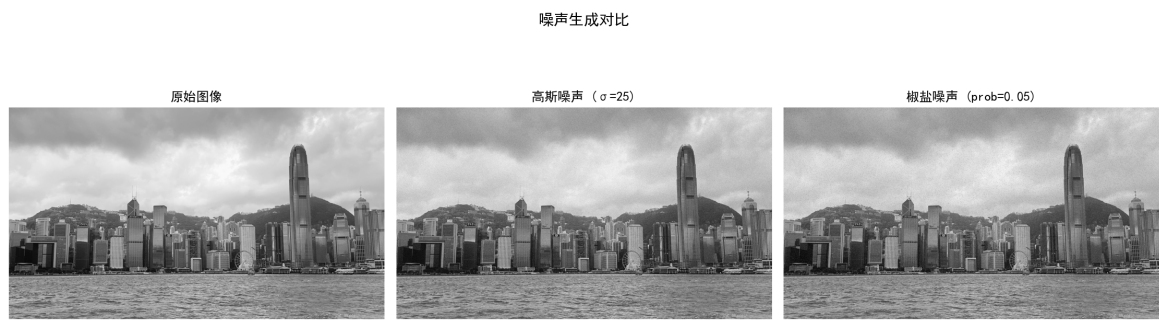


图 2: 原始图像与两种噪声对比（高斯噪声 $\sigma = 25$ ，椒盐噪声 $p = 0.05$ ）

结果分析：图 2 展示了两类噪声的视觉效果差异：

1. **高斯噪声（中图）**：图像呈现均匀的”雾状”颗粒感。因为高斯噪声影响每一个像素（只是幅度不同），整体画面被一层随机亮度波动覆盖。 $\sigma = 25$ 的强度在 255 灰

度级上对应约 10% 的波动，主观感受为明显的” 胶片颗粒” 效果，但画面内容仍然可辨。

- 2. **椒盐噪声（右图）**：表现为随机散布的黑白像素点。 $p = 0.05$ 即每 20 个像素中约有 1 个被替换为纯黑或纯白。与高斯噪声相比，椒盐噪声的视觉干扰更加” 突兀”——单个噪点的对比度极高，对视觉注意力的干扰更强。
- 3. **退化机理差异**：高斯噪声属于” 加性噪声”（additive），每个像素都受影响；椒盐噪声属于” 替换噪声”（replacement），仅影响少数像素但影响极端。这一本质区别决定了后续去噪策略的选择。

4.3 空间域平滑滤波



图 3: 原始图像与均匀平滑、高斯平滑对比（5×5 核）

结果分析：图 3 对比了两种线性平滑滤波器的效果：

- 1. **均匀平滑（中图）**：图像整体明显模糊，边缘和纹理区域被” 钝化”。因为 5×5 矩形窗内所有 25 个像素权重相同——边缘另一侧的像素也被同等纳入均值计算——导致跨边缘的亮度过渡被拉平。在面部的高频区域（如眼睛、嘴唇边缘），细节损失较为严重。
- 2. **高斯平滑（右图）**：同样存在模糊，但边缘保留略优于均匀平滑。高斯核的中心像素权重最大（ $\sigma = 1.0$ 时中心权重约为边缘权重的 $e^8 \approx 2981$ 倍），因此输出像素更依赖于邻近的同类像素而非跨边缘的异类像素。
- 3. **定量差异**：两种平滑在视觉上差异不大（ $\sigma = 1.0$ 的高斯核与同等大小的均匀核在此尺度下近似），区分主要在于大核（ $K \geq 7$ ）或大 σ 场景下高斯平滑的优越性更加明显。

4.4 去噪处理



图 4: 去噪效果对比（上排：高斯噪声去噪；下排：椒盐噪声去噪）

结果分析：图 4 是本次实验最核心的对比结果，揭示了线性与非线性滤波在不同噪声模型下的根本差异。

高斯噪声去噪（上排）

- **高斯平滑去噪（中上）：**效果较好。由于高斯噪声影响每个像素，线性加权平均能够有效“稀释”噪声方差—— $K \times K$ 核的均值操作将噪声方差降低为原来的 $1/K^2 = 1/25$ 。缺点是边缘和纹理被同时平滑，图像整体偏柔和。
- **中值滤波去噪（右上）：**效果不理想。高斯噪声使每个像素都偏离原值，取中位数并不能有效区分“噪声像素”和“正常像素”。残留噪声较多，且由于中值滤波的非线性特性，在平坦区域可能出现轻微的“斑块状”伪影（patchy artifacts）。
- **结论：**对于高斯噪声，高斯平滑 $\gg$ 中值滤波。

椒盐噪声去噪（下排）

- **高斯平滑去噪（中下）：**效果极差。线性卷积将纯黑（0）或纯白（255）的噪点“涂抹”到周围 $5 \times 5 = 25$ 个像素上，形成灰黑色“污渍”。这是因为线性滤波无法区分噪声像素和正常像素——当核窗口覆盖到噪点时，该极端值直接污染了整个邻域的加权均值。

- **中值滤波去噪（右下）：**效果卓越。椒盐噪声取值为 0 或 255（极端值），在排序后的 25 个像素中必然位于两端，中位数取到的一定是正常像素值。噪声被精准剔除，且由于中值滤波的非线性特性，跨边缘的像素值不会被平均化——只要边缘两侧像素各占邻域一半以上，中位数就落在正确的一侧，从而保持边缘的锐利。
- **结论：**对于椒盐噪声，中值滤波 $\gg$ 高斯平滑。

4.5 综合分析



图 5: 线性滤波综合分析（3 行 ×4 列子图）

图 5 将所有实验结果整合到一张 3×4 布局的大图中，便于跨方法、跨噪声类型的系统对比。第一行展示平滑滤波的基线效果，第二行和第三行分别展示高斯噪声和椒盐噪声的去噪对比。第 3 列故意留空，在视觉上将”平滑”与”去噪”两组实验分隔开来。

5 讨论

5.1 线性 vs 非线性滤波

本实验最核心的发现是线性滤波与非线性滤波在适用场景上的根本差异：

表 3: 滤波器适用性对比

滤波器	高斯噪声	椒盐噪声
高斯平滑（线性）	好	差（产生伪影）
中值滤波（非线性）	一般	优秀

这一差异来源于两类滤波器的数学本质：

- **线性滤波**对所有像素一视同仁地加权平均——在噪声普遍存在时（高斯噪声）有效降低方差，但在仅有少数极端噪声时（椒盐噪声），极端值会污染整个邻域。
- **非线性滤波**通过排序操作选择性忽略极端值——对脉冲噪声天然免疫，但在噪声普遍存在时无法区分信号与噪声。

5.2 噪声抑制与边缘保留的权衡

所有空间域滤波都面临一个根本性的权衡（trade-off）：更大的核 = 更强的去噪能力 + 更严重的边缘模糊。这一现象的数学解释为：

- 对于线性滤波，核大小 K 与噪声方差缩减比例为 $1/K^2$ ，但同时核的”空间扩散”范围也以 $O(K^2)$ 增长，导致跨边缘污染加剧。
- 中值滤波在此权衡上表现更好——因为中位数操作具有”边缘保持”（edge-preserving）特性，只要边缘两侧像素在邻域中各占一半以上，中位数就不会跨过边缘。

实际应用中，通常需要根据噪声类型和图像内容选择合适的滤波器及参数。更高级的方法（如双边滤波 Bilateral Filter、非局部均值 NLM、BM3D）在噪声抑制和边缘保留之间取得了更好的平衡，但计算复杂度相应增加。

5.3 参数敏感性分析

- **高斯噪声 σ** ： σ 越大，噪声越强，去噪难度越大。 $\sigma = 25$ 为中等水平， $\sigma > 50$ 时高斯平滑仍有效但需增大核大小（如 7×7 或 9×9 ）。

- **椒盐噪声 p** : p 越大，噪声密度越高。中值滤波在 $p < 0.5$ 时理论上有良好的去噪效果（因为“好”像素仍占多数），但 p 接近 0.5 时性能急剧下降。本实验的 $p = 0.05$ 在 5×5 核（25 个像素）下最多期望覆盖 1-2 个噪点，中值滤波能轻松处理。
- **核大小 K** : $K = 5$ 是平滑与细节保留之间的折中选择。 $K = 3$ 去噪不足， $K = 7$ 过度平滑。

5.4 实验局限与改进方向

1. **客观评价指标缺失**: 本实验仅采用主观视觉评价，未使用 PSNR（峰值信噪比）或 SSIM（结构相似性）等定量指标。在有原始无噪声图像（ground truth）的情况下，可使用 PSNR 和 SSIM 实现客观的数值对比。
2. **参数范围有限**: 仅测试了单一参数组合（ $\sigma = 25, p = 0.05, K = 5$ ），未进行系统的参数扫描实验。
3. **滤波器种类有限**: 未纳入自适应中值滤波、双边滤波等更先进的去噪方法作为对比基线。
4. **单张测试图像**: 仅使用一张照片，未考察滤波器在不同图像内容（纹理丰富 vs 平坦、高对比度 vs 低对比度）下的表现差异。

6 总结

本实验通过 Python + OpenCV 实现了图像线性滤波的完整流程，涵盖了高斯金字塔构建、噪声生成、空间域平滑滤波和去噪处理五大模块，并使用真实照片进行了系统的对比实验。

核心发现：

1. 高斯金字塔通过反复的高斯平滑 + 下采样实现多尺度表示，每层分辨率减半，信息量逐层递减。
2. 高斯噪声（加性噪声）和椒盐噪声（脉冲噪声）是两种完全不同的退化模型：前者影响所有像素（幅度分布），后者仅影响少数像素（空间随机）。
3. 线性滤波器（均匀平滑、高斯平滑）能有效抑制高斯噪声（方差降低 $1/K^2$ ），但对椒盐噪声无效，甚至因“涂抹”噪点而产生视觉伪影。
4. 非线性中值滤波对椒盐噪声有卓越的去噪效果——利用排序操作将极端值天然排斥在中位数之外，同时保持边缘锐利。
5. 空间域滤波的核心挑战是“噪声抑制 vs 边缘保留”的权衡——所有方法都在这两个目标之间寻求平衡，不存在普适的最优解。

本实验为后续学习更高级的图像处理技术（如频域滤波、小波去噪、深度学习去噪）奠定了理论与实验基础。

A 完整源代码 (main.py)

以下为实验完整源代码，包含所有模块的实现与注释。

Listing 9: main.py ——线性滤波实验完整代码

```
1 """
2 CV课程作业（1）：线性滤波
3 实现：高斯金字塔、噪声生成、均匀平滑、高斯平滑、中值滤波
4 """
5
6 import cv2
7 import numpy as np
8 from matplotlib import pyplot as plt
9 from matplotlib import rcParams
10
11 # 中文字体配置
12 rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu
    Sans']
13 rcParams['axes.unicode_minus'] = False
14
15
16 def load_image(path, gray=True):
17     """加载图像，可选转为灰度图"""
18     img = cv2.imread(path)
19     if img is None:
20         raise FileNotFoundError(f"无法加载图像: {path}")
21     img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
22     if gray:
23         img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
24     return img
25
26
27 # ===== 1. 高斯金字塔 =====
28
29 def gaussian_pyramid(img, levels=4):
30     """
31     构建高斯金字塔
32     每一层通过高斯模糊 + 下采样得到
33     """
34     pyramid = [img]
35     current = img.copy()
36     for i in range(levels - 1):
37         current = cv2.pyrDown(current)
38         pyramid.append(current)
```

```
39     return pyramid
40
41
42 def show_pyramid(pyramid):
43     """子图显示高斯金字塔"""
44     n = len(pyramid)
45     fig, axes = plt.subplots(1, n, figsize=(4 * n, 4))
46     for i, (ax, img) in enumerate(zip(axes, pyramid)):
47         ax.imshow(img, cmap='gray')
48         ax.set_title(f'第{i}层 ({img.shape[1]}x{img.shape[0]}),
49                     fontsize=11)
50         ax.axis('off')
51     fig.suptitle('高斯金字塔 (Gaussian Pyramid)', fontsize=14,
52                 fontweight='bold')
53     plt.tight_layout()
54     return fig
55
56 # ===== 2. 噪声生成 =====
57
58 def add_gaussian_noise(img, mean=0, sigma=25):
59     """添加高斯噪声"""
60     noise = np.random.normal(mean, sigma, img.shape).astype(np.float32)
61     )
62     noisy = img.astype(np.float32) + noise
63     noisy = np.clip(noisy, 0, 255).astype(np.uint8)
64     return noisy
65
66
67 def add_salt_pepper_noise(img, prob=0.05):
68     """添加椒盐噪声"""
69     noisy = img.copy()
70     h, w = img.shape
71     n_pixels = h * w
72
73     # 盐噪声 (白色)
74     n_salt = int(n_pixels * prob / 2)
75     salt_coords = (np.random.randint(0, h, n_salt),
76                   np.random.randint(0, w, n_salt))
77     noisy[salt_coords] = 255
78
79     # 椒噪声 (黑色)
80     n_pepper = int(n_pixels * prob / 2)
81     pepper_coords = (np.random.randint(0, h, n_pepper),
82                     np.random.randint(0, w, n_pepper))
```

```
81     noisy[pepper_coords] = 0
82
83     return noisy
84
85
86 # ===== 3. 平滑滤波 =====
87
88 def uniform_smoothing(img, kernel_size=5):
89     """均匀平滑（均值滤波）"""
90     return cv2.blur(img, (kernel_size, kernel_size))
91
92
93 def gaussian_smoothing(img, kernel_size=5, sigma=1.0):
94     """高斯平滑"""
95     return cv2.GaussianBlur(img, (kernel_size, kernel_size), sigma)
96
97
98 def median_filtering(img, kernel_size=5):
99     """中值滤波"""
100    return cv2.medianBlur(img, kernel_size)
101
102
103 # ===== 结果可视化 =====
104
105 def show_noise_results(original, gaussian_noisy, sp_noisy):
106     """显示原始图像和加噪结果"""
107     fig, axes = plt.subplots(1, 3, figsize=(15, 5))
108
109     images = [
110         (original, '原始图像'),
111         (gaussian_noisy, '高斯噪声 (=25)'),
112         (sp_noisy, '椒盐噪声 (prob=0.05)'),
113     ]
114     for ax, (img, title) in zip(axes, images):
115         ax.imshow(img, cmap='gray')
116         ax.set_title(title, fontsize=12)
117         ax.axis('off')
118     fig.suptitle('噪声生成对比', fontsize=14, fontweight='bold')
119     plt.tight_layout()
120     return fig
121
122
123 def show_smoothing_results(original, uniform, gaussian):
124     """显示原图及平滑结果"""
125     fig, axes = plt.subplots(1, 3, figsize=(15, 5))
```



```
126
127     images = [
128         (original, '原始图像'),
129         (uniform, '均匀平滑 (5x5)'),
130         (gaussian, '高斯平滑 (5x5,  $\sigma=1$ )'),
131     ]
132     for ax, (img, title) in zip(axes, images):
133         ax.imshow(img, cmap='gray')
134         ax.set_title(title, fontsize=12)
135         ax.axis('off')
136     fig.suptitle('空间域平滑滤波', fontsize=14, fontweight='bold')
137     plt.tight_layout()
138     return fig
139
140
141 def show_denoising_results(gaussian_noisy, gs_denoised,
142                           median_denoised_g,
143                           sp_noisy, gs_sp_denoised,
144                           median_denoised_sp):
145     """显示去噪结果对比（2行3列）"""
146     fig, axes = plt.subplots(2, 3, figsize=(15, 10))
147
148     # 第一行：高斯噪声去噪
149     for ax, (img, title) in zip(axes[0], [
150         (gaussian_noisy, '高斯噪声图像'),
151         (gs_denoised, '高斯平滑去噪 (5x5)'),
152         (median_denoised_g, '中值滤波去噪 (5x5)'),
153     ]):
154         ax.imshow(img, cmap='gray')
155         ax.set_title(title, fontsize=11)
156         ax.axis('off')
157
158     # 第二行：椒盐噪声去噪
159     for ax, (img, title) in zip(axes[1], [
160         (sp_noisy, '椒盐噪声图像'),
161         (gs_sp_denoised, '高斯平滑去噪 (5x5)'),
162         (median_denoised_sp, '中值滤波去噪 (5x5)'),
163     ]):
164         ax.imshow(img, cmap='gray')
165         ax.set_title(title, fontsize=11)
166         ax.axis('off')
167
168     fig.suptitle('噪声去除效果对比', fontsize=14, fontweight='bold')
169     plt.tight_layout()
170     return fig
```

```
169
170
171 # ===== 主 流 程 =====
172
173 def main(image_path):
174     # 加载图像
175     img = load_image(image_path, gray=True)
176     print(f"图像尺寸: {img.shape}")
177
178     # ---- 1. 高斯金字塔 ----
179     pyramid = gaussian_pyramid(img, levels=4)
180     fig1 = show_pyramid(pyramid)
181     fig1.savefig('output_1_pyramid.png', dpi=150, bbox_inches='tight')
182     print("1/5 高斯金字塔完成")
183
184     # ---- 2. 添加噪声 ----
185     img_gaussian_noise = add_gaussian_noise(img, sigma=25)
186     img_sp_noise = add_salt_pepper_noise(img, prob=0.05)
187     fig2 = show_noise_results(img, img_gaussian_noise, img_sp_noise)
188     fig2.savefig('output_2_noise.png', dpi=150, bbox_inches='tight')
189     print("2/5 噪声生成完成")
190
191     # ---- 3. 平滑滤波 ----
192     img_uniform = uniform_smoothing(img, kernel_size=5)
193     img_gaussian_smooth = gaussian_smoothing(img, kernel_size=5, sigma
194                                             =1.0)
195     fig3 = show_smoothing_results(img, img_uniform,
196                                   img_gaussian_smooth)
197     fig3.savefig('output_3_smoothing.png', dpi=150, bbox_inches='tight')
198     print("3/5 空间域平滑完成")
199
200     # ---- 4. 噪声图像去噪 ----
201     # 高斯噪声 → 高斯平滑
202     gs_on_gaussian = gaussian_smoothing(img_gaussian_noise,
203                                         kernel_size=5, sigma=1.0)
204     # 高斯噪声 → 中值滤波
205     median_on_gaussian = median_filtering(img_gaussian_noise,
206                                           kernel_size=5)
207     # 椒盐噪声 → 高斯平滑
208     gs_on_sp = gaussian_smoothing(img_sp_noise, kernel_size=5, sigma
209                                   =1.0)
210     # 椒盐噪声 → 中值滤波
211     median_on_sp = median_filtering(img_sp_noise, kernel_size=5)
```

```
208     fig4 = show_denoising_results(  
209         img_gaussian_noise, gs_on_gaussian, median_on_gaussian,  
210         img_sp_noise, gs_on_sp, median_on_sp  
211     )  
212     fig4.savefig('output_4_denoising.png', dpi=150, bbox_inches='tight'  
213         ')  
214     print("4/5 去噪处理完成")  
215  
216     # ---- 5. 综合分析 ----  
217     fig5 = comprehensive_analysis(img, img_gaussian_noise,  
218         img_sp_noise,  
219         img_uniform, img_gaussian_smooth,  
220         gs_on_gaussian, median_on_gaussian,  
221         gs_on_sp, median_on_sp)  
222     fig5.savefig('output_5_analysis.png', dpi=150, bbox_inches='tight'  
223         )  
224     print("5/5 综合分析完成")  
225  
226     plt.show()  
227     print("所有结果已保存为 PNG 文件")  
228  
229  
230 def comprehensive_analysis(original, gauss_noisy, sp_noisy,  
231     uniform, gauss_smooth,  
232     gs_on_gauss, median_on_gauss,  
233     gs_on_sp, median_on_sp):  
234     """综合分析大图"""  
235     fig = plt.figure(figsize=(18, 12))  
236  
237     positions = [  
238         (original, '原始图像', 0),  
239         (uniform, '均匀平滑 (5x5)', 1),  
240         (gauss_smooth, '高斯平滑 (5x5,  $\sigma=1$ )', 2),  
241         (gauss_noisy, '高斯噪声 ( $p=0.05$ )', 3),  
242         (gs_on_gauss, '高斯平滑去噪', 4),  
243         (median_on_gauss, '中值滤波去噪', 5),  
244         (sp_noisy, '椒盐噪声 ( $p=0.05$ )', 6),  
245         (gs_on_sp, '高斯平滑去噪', 7),  
246         (median_on_sp, '中值滤波去噪', 8),  
247     ]  
248  
249     for img, title, idx in positions:  
250         ax = fig.add_subplot(3, 4, idx + 1)  
251         ax.imshow(img, cmap='gray')  
252         ax.set_title(title, fontsize=10)
```

```

250     ax.axis('off')
251
252     # 添加行标签
253     fig.text(0.02, 0.78, '平滑滤波', fontsize=12, fontweight='bold',
254             va='center', rotation=90)
255     fig.text(0.02, 0.50, '高斯噪声\n去噪对比', fontsize=12, fontweight=
256             = 'bold',
257             va='center', rotation=90)
258     fig.text(0.02, 0.22, '椒盐噪声\n去噪对比', fontsize=12, fontweight
259             = 'bold',
260             va='center', rotation=90)
261
262     fig.suptitle('线性滤波综合分析', fontsize=16, fontweight='bold')
263     plt.tight_layout(rect=[0.05, 0, 1, 0.96])
264     return fig
265
266 if __name__ == '__main__':
267     import sys
268     if len(sys.argv) > 1:
269         path = sys.argv[1]
270     else:
271         # 默认：使用 OpenCV 内置示例图
272         path = cv2.samples.findFile('lena.jpg')
273         if path is None:
274             print("请提供图像路径：python main.py <image_path>")
275             print("或放置 lena.jpg 到当前目录")
276             sys.exit(1)
277     main(path)

```

B 输出文件清单

表 4: 实验输出文件

文件名	内容	格式
output_1_pyramid.png	4 层高斯金字塔	1276×3408, 灰度
output_2_noise.png	原始图像 + 两种噪声对比	1276×3828, 灰度
output_3_smoothing.png	原始 + 均匀平滑 + 高斯平滑	1276×3828, 灰度
output_4_denoising.png	2×3 去噪对比矩阵	2552×3828, 灰度
output_5_analysis.png	3×4 综合分析大图	3828×5104, 灰度

C 运行说明

在项目目录下执行以下命令即可复现所有实验结果：

Listing 10: 运行命令

```
1 pip install -r requirements.txt
2 python main.py input_photo.jpg
```

若需使用其他图像，将 `input_photo.jpg` 替换为目标图像路径即可。生成的 5 张 PNG 输出图将保存在当前目录。